

kuramoto-lean: A Sorry-Free Lean 4 Library for Finite-N Kuramoto Synchronisation Dynamics

Ben Cassie
Independent Researcher
bencassie@outlook.com
ORCID: 0009-0004-1899-7627

Published on Zenodo. DOI: 10.5281/zenodo.20468619.

Abstract

We present `kuramoto-lean`, a Lean 4 / Mathlib library containing 14 formally verified theorem and lemma statements about finite- N Kuramoto oscillator dynamics. The library takes a finite-dimensional geometric approach on phase configurations, distinct from the Ott–Antonsen manifold and continuum-limit formalism used in complementary Kuramoto formalisation work. All public results in the library are sorry-free, admit-free, and introduce no new axioms. We describe the library contents and the `#check-before-cite` development discipline used to avoid hallucinated API names during AI-assisted Lean development.

1. Introduction

The Kuramoto model is one of the standard mathematical models of synchronisation. It describes a population of coupled phase oscillators, each with phase θ_i , whose velocities depend on phase differences with other oscillators. In its identical-frequency, all-to-all form, the model is often written

$$\dot{\theta}_i = \frac{K}{N} \sum_{j=1}^N \sin(\theta_j - \theta_i),$$

where K is a global coupling strength. Positive coupling drives phases toward agreement, and the order parameter

$$R(\theta) = \frac{1}{N} \sum_{j=1}^N e^{i\theta_j}$$

measures the degree of coherence. The model and its variants appear in biological synchrony, neural dynamics, power-grid stability, distributed timing, and oscillator-based computing.

This paper is not a claim of new dynamical-systems theory. The mathematical facts formalised here are standard finite-dimensional identities or algebraic consequences of standard definitions. The contribution is instead a machine-checked proof artifact: a Lean 4 / Mathlib library in which these identities are

represented precisely, compiled against a pinned toolchain, and verified without `sorry`, `admit`, or additional axioms. For dynamical systems, this distinction matters. Informal calculations involving finite sums, signs, derivatives, and symmetries are easy to state but can hide mismatched conventions or incorrect factors. A formal library exposes these choices explicitly.

The library differs in scope from `taejun-song/kuramoto-lean`, which develops Kuramoto stability results through continuum and Ott–Antonsen machinery. In a local inspection of that repository, files such as `Complex0A.lean`, `ContinuumRigidity.lean`, and `KuramotoFinal.lean` show the continuum/Ott–Antonsen orientation. By contrast, `kuramoto-lean` works directly with finite index types `Fin N`, finite sums, explicit vector fields, and coordinate derivatives. Its main focus is finite- N geometric structure: order-parameter boundedness, gradient identities, pairwise contraction contributions, weighted Lyapunov descent, and Hebbian phase-plus-weight descent.

The library also serves as the formal proof spine for the companion `flywheel-universe` project, which studies budgeted Hebbian Kuramoto dynamics for Max-Cut under coupling-resource constraints. The companion project supplies the experimental model, browser demo, and benchmark context; this library supplies a sorry-free formalisation of the unprojected algebraic descent core. The companion paper is available at the Zenodo record <https://zenodo.org/records/20303914>. The present library does not formalise the full projected constrained dynamics, projected KKT stationarity, or trajectory-level convergence claims from that project.

The remainder of the paper is organised as follows. Section 2 describes the library module by module. Section 3 records the verified artifact and reproducibility information. Section 4 describes the development methodology, especially the `#check-before-cite` discipline. Section 5 situates the library relative to the companion project, witness-theory framing, ATLAS, and related Kuramoto formalisation work. Section 6 states the limitations and future work. Section 7 concludes.

2. The Library

The repository consists of a root import file, `Kuramoto.lean`, and seven theorem-bearing modules under `Kuramoto/`. The public result set contains 14 theorem or lemma statements. The following catalogue summarises the results without relying on a wide table, so that the rendered PDF remains readable.

- **Order parameter bound** (`kuramotoR_norm_le_one`, `OrderParameter.lean`): for $N > 0$, $\|R_N(\theta)\| \leq 1$. This is the basic geometric bound for finite phasor averages.
- **Uniform gradient identity** (`kuramoto_gradient_identity`, `GradientFlow.lean`): $F_i(\theta) = -\partial_i V(\theta)$. This identifies the uniform identical Kuramoto field as a negative gradient flow.

- **$N = 2$ relative velocity** (`kuramoto_relative_velocity`, `Contraction.lean`): $F_0 - F_1 = -K \sin(\theta_0 - \theta_1)$. This gives exact two-oscillator relative dynamics.
- **$N = 2$ pairwise contraction** (`kuramoto_pairwise_contraction`, `Contraction.lean`): if $K > 0$ and the phase gap lies in $(0, \pi)$, then $F_0 < F_1$. The leading oscillator slows relative to the trailing one.
- **General direct-coupling contraction** (`kuramoto_coupling_contraction`, `Contraction.lean`): a positive symmetric direct coupling contributes negatively to relative velocity. This is a general- N pairwise result, not a global synchronisation theorem.
- **Weighted gradient identity** (`weighted_gradient_identity`, `Weighted.lean`): for symmetric W , $F_i^W = -\partial_i V_W$. Weighted finite networks retain gradient structure.
- **Symmetry cancellation** (`weightedKuramotoV_symm_cancel`, `Weighted.lean`): $\sum_i F_i^2 = -\sum_i F_i \partial_i V_W$. This is the algebraic cancellation underlying descent.
- **Weighted Lyapunov descent** (`weighted_lyapunov_descent`, `Weighted.lean`): $\sum_i F_i \partial_i V_W \leq 0$. The potential decreases in the vector-field direction.
- **Hebbian weight gradient identity** (`hebbian_weight_gradient_identity`, `Hebbian.lean`): $G_{ij} = -\partial_{W_{ij}} L$. The weight update is the negative gradient of the joint Lyapunov function.
- **Hebbian phase gradient identity** (`hebbian_phase_gradient_identity`, `Hebbian.lean`): $F_i^W = -\partial_i L$. The phase update is the negative gradient of the same joint Lyapunov function.
- **Joint Hebbian descent** (`hebbian_joint_lyapunov_descent`, `Hebbian.lean`): phase and weight terms jointly contribute a non-positive directional derivative. This is the unprojected algebraic joint descent core.
- **Zero-regularisation connection** (`hebbianL_zero_lam_eq_weighted`, `Connections.lean`): $L_{K,0} = V_W$. The Hebbian Lyapunov function generalises the weighted Kuramoto potential.
- **Weight-entry convexity** (`hebbianL_convex_weight_entry`, `Connections.lean`): $\partial^2 L / \partial W_{ij}^2 > 0$ when $\lambda > 0$. The quadratic regulariser gives entrywise strict convexity.
- **Barrier asymmetry toy result** (`barrier_asymmetry_direct`, `WitnessGeometry.lean`): unequal quadratic curvatures give unequal restoring-force magnitudes. This is a minimal witness-geometry force-asymmetry formalisation.

2.1 OrderParameter.lean

For N oscillators with phases $\theta : \text{Fin } N \rightarrow \mathbb{R}$, the library defines the complex Kuramoto order parameter

$$R_N(\theta) = \frac{1}{N} \sum_{k: \text{Fin } N} \exp(i\theta_k).$$

The theorem states that the norm of this average of unit phasors is bounded by one whenever $N > 0$.

```
noncomputable def kuramotoR (N : ℕ) (θ : Fin N → ℝ) : ℂ :=
  (∑ k, Complex.exp (θ k * Complex.I)) / N
```

```
theorem kuramotoR_norm_le_one (N : ℕ) (hN : 0 < N) (θ : Fin N → ℝ) :
  ||kuramotoR N θ|| ≤ 1
```

The proof is the triangle inequality for a finite sum in $\mathbb{C}$. Each summand has norm one, established through Mathlib's circle exponential interface (`Circle.coe_exp` and `Circle.norm_coe`). The norm of the sum is therefore at most N , and division by the positive real coercion of N gives the result.

2.2 GradientFlow.lean

For the identical all-to-all Kuramoto model, the library defines the potential

$$V_K(\theta) = -\frac{K}{2N} \sum_i \sum_j \cos(\theta_j - \theta_i)$$

and vector field

$$F_i(\theta) = \frac{K}{N} \sum_j \sin(\theta_j - \theta_i).$$

The main theorem states that each vector-field component is the negative coordinate derivative of the potential.

```
noncomputable def kuramotoV (K : ℝ) (N : ℕ) (θ : Fin N → ℝ) : ℝ :=
  -(K / (2 * N)) * ∑ i : Fin N, ∑ j : Fin N, Real.cos (θ j - θ i)
```

```
noncomputable def kuramotoF (K : ℝ) (N : ℕ) (i : Fin N) (θ : Fin N → ℝ) : ℝ :=
  (K / N) * ∑ j : Fin N, Real.sin (θ j - θ i)
```

```
theorem kuramoto_gradient_identity
  (K : ℝ) (N : ℕ) (hN : 0 < N) (hK : 0 < K)
  (θ : Fin N → ℝ) (i : Fin N) :
  kuramotoF K N i θ =
    -(deriv (fun x => kuramotoV K N (Function.update θ i x)) (θ i))
```

The proof differentiates a finite double sum term by term. Updating the i -th phase coordinate is represented by `Function.update θ i x`. The derivative of

each cosine term is encoded by a local `pairDeriv`, and a finite-sum identity collapses the double sum to $2 \sum_j \sin(\theta_j - \theta_i)$. The factor 2 cancels the $1/(2N)$ in the potential, yielding the vector field.

2.3 Contraction.lean

The first contraction result specialises the uniform vector field to $N = 2$. It proves the exact relative-velocity identity

$$F_0(\theta) - F_1(\theta) = -K \sin(\theta_0 - \theta_1).$$

```
lemma kuramoto_relative_velocity
  (K : ℝ) (θ : Fin 2 → ℝ) :
    kuramotoF K 2 0 θ - kuramotoF K 2 1 θ =
      -K * Real.sin (θ 0 - θ 1)
```

The proof expands the two finite sums using `Fin.sum_univ_two`, eliminates diagonal sine terms with $\sin 0 = 0$, and rewrites $\sin(\theta_1 - \theta_0)$ using oddness of sine.

The second result converts the identity into a strict contraction inequality. If $K > 0$ and the phase gap lies in $(0, \pi)$, then $\sin(\theta_0 - \theta_1) > 0$, so $F_0 - F_1 < 0$.

```
theorem kuramoto_pairwise_contraction
  (K : ℝ) (hK : 0 < K) (θ : Fin 2 → ℝ)
  (hgap : 0 < θ 0 - θ 1) (hpi : θ 0 - θ 1 < Real.pi) :
    kuramotoF K 2 0 θ < kuramotoF K 2 1 θ
```

The third result avoids claiming full general- N synchronisation. Instead, it isolates the direct pairwise coupling contribution in a weighted system. For symmetric W , positive W_{ij} , and a gap in $(0, \pi)$, the direct contribution to $F_i - F_j$ is strictly negative:

$$KW_{ij} \sin(\theta_j - \theta_i) - KW_{ji} \sin(\theta_i - \theta_j) < 0.$$

```
theorem kuramoto_coupling_contraction
  (K : ℝ) (hK : 0 < K)
  (N : ℕ) (W : Fin N → Fin N → ℝ) (hW : ∀ i j, W i j = W j i)
  (θ : Fin N → ℝ) (i j : Fin N)
  (hWpos : 0 < W i j)
  (hgap : 0 < θ i - θ j) (hpi : θ i - θ j < Real.pi) :
    K * W i j * Real.sin (θ j - θ i)
      - K * W j i * Real.sin (θ i - θ j) < 0
```

This theorem is deliberately local. In a general network, other oscillators also contribute to $F_i - F_j$, so the theorem does not assert global contraction. It asserts that the direct positive symmetric pair coupling has the expected sign.

2.4 Weighted.lean

The weighted module generalises the uniform all-to-all potential to an arbitrary symmetric coupling matrix $W : \text{Fin } N \rightarrow \text{Fin } N \rightarrow \mathbb{R}$:

$$V_W(\theta) = -\frac{K}{2} \sum_i \sum_j W_{ij} \cos(\theta_j - \theta_i),$$

with vector field

$$F_i^W(\theta) = K \sum_j W_{ij} \sin(\theta_j - \theta_i).$$

```

noncomputable def weightedKuramotoV (K : ℝ) (N : ℕ) (W : Fin N → Fin N → ℝ)
  (θ : Fin N → ℝ) : ℝ :=
  -(K / 2) * ∑ i : Fin N, ∑ j : Fin N, W i j * Real.cos (θ j - θ i)

noncomputable def weightedKuramotoF (K : ℝ) (N : ℕ) (W : Fin N → Fin N → ℝ)
  (i : Fin N) (θ : Fin N → ℝ) : ℝ :=
  K * ∑ j : Fin N, W i j * Real.sin (θ j - θ i)

lemma weighted_gradient_identity
  (K : ℝ) (N : ℕ) (W : Fin N → Fin N → ℝ) (hW : ∀ i j, W i j = W j i)
  (θ : Fin N → ℝ) (i : Fin N) :
  weightedKuramotoF K N W i θ =
    -(deriv (fun x => weightedKuramotoV K N W (Function.update θ i x)) (θ i))

```

The symmetry assumption is essential. Without $W_{ij} = W_{ji}$, differentiating the full double sum produces the symmetrised coupling rather than the stated vector field. The formal proof repeats the finite-sum derivative structure from the uniform case, but the collapse lemma uses symmetry to combine the W_{ij} and W_{ji} terms.

The module then proves an algebraic cancellation identity and a descent inequality:

```

lemma weightedKuramotoV_symm_cancel
  (K : ℝ) (N : ℕ) (W : Fin N → Fin N → ℝ) (hW : ∀ i j, W i j = W j i)
  (θ : Fin N → ℝ) :
  ∑ i : Fin N, weightedKuramotoF K N W i θ * weightedKuramotoF K N W i θ =
    -(∑ i : Fin N, weightedKuramotoF K N W i θ *
      deriv (fun x => weightedKuramotoV K N W (Function.update θ i x)) (θ i))

theorem weighted_lyapunov_descent
  (K : ℝ) (_hK : 0 < K) (N : ℕ) (W : Fin N → Fin N → ℝ) (hW : ∀ i j, W i j = W j i)
  (θ : Fin N → ℝ) :
  ∑ i : Fin N, weightedKuramotoF K N W i θ *
    deriv (fun x => weightedKuramotoV K N W (Function.update θ i x)) (θ i) ≤ 0

```

Mathematically, substituting $F_i^W = -\partial_i V_W$ gives

$$\sum_i F_i^W \partial_i V_W = -\sum_i (F_i^W)^2 \leq 0.$$

This is an algebraic directional-descent statement, not a theorem about ODE solutions.

2.5 Hebbian.lean

The Hebbian module introduces a joint phase-and-weight Lyapunov function

$$L_{K,\lambda}(\theta, W) = V_W(\theta) + \frac{\lambda}{2} \sum_i \sum_j W_{ij}^2,$$

and an unprojected Hebbian weight field

$$G_{ij}(\theta, W) = \frac{K}{2} \cos(\theta_j - \theta_i) - \lambda W_{ij}.$$

The coordinate update helper `hebbianUpdateWeight W i j x` replaces the single weight entry W_{ij} with x .

```

noncomputable def hebbianL (K lam : ℝ) (N : ℕ) (W : Fin N → Fin N → ℝ)
  (θ : Fin N → ℝ) : ℝ :=
  weightedKuramotoV K N W θ + (lam / 2) * ∑ i : Fin N, ∑ j : Fin N, W i j * W i j

noncomputable def hebbianWeightF (K lam : ℝ) (N : ℕ) (W : Fin N → Fin N → ℝ)
  (θ : Fin N → ℝ) (i j : Fin N) : ℝ :=
  (K / 2) * Real.cos (θ j - θ i) - lam * W i j

theorem hebbian_weight_gradient_identity
  (K lam : ℝ) (N : ℕ) (W : Fin N → Fin N → ℝ)
  (θ : Fin N → ℝ) (i j : Fin N) :
  hebbianWeightF K lam N W θ i j =
    -(deriv (fun x => hebbianL K lam N (hebbianUpdateWeight W i j x) θ) (W i j))

The proof differentiates  $L$  with respect to a single weight coordinate. The poten-
tial part contributes  $-(K/2) \cos(\theta_j - \theta_i)$ ; the quadratic regulariser contributes
 $\lambda W_{ij}$ . Negating their sum gives the stated weight field.

The phase-side theorem states that, for symmetric  $W$ , the weighted phase vector
field is also the negative coordinate derivative of the same joint Lyapunov
function:

theorem hebbian_phase_gradient_identity
  (K lam : ℝ) (N : ℕ) (W : Fin N → Fin N → ℝ) (hW : ∀ i j, W i j = W j i)
  (θ : Fin N → ℝ) (i : Fin N) :
```

```

weightedKuramotoF K N W i θ =
  -(deriv (fun x => hebbianL K lam N W (Function.update θ i x)) (θ i))

```

The quadratic weight regulariser is constant with respect to phase coordinates, so the result reduces to the weighted gradient identity.

The central Hebbian result is the joint algebraic descent theorem:

```

theorem hebbian_joint_lyapunov_descent
  (K lam : ℝ) (N : ℕ) (W : Fin N → Fin N → ℝ) (hW : ∀ i j, W i j = W j i)
  (θ : Fin N → ℝ) :
  (∑ i : Fin N, weightedKuramotoF K N W i θ *
    deriv (fun x => hebbianL K lam N W (Function.update θ i x)) (θ i))
  + (∑ i : Fin N, ∑ j : Fin N, hebbianWeightF K lam N W θ i j *
    deriv (fun x => hebbianL K lam N (hebbianUpdateWeight W i j x) θ) (W i j)) ≤ 0

```

The theorem says that the phase contribution and weight contribution together are non-positive. Substituting the two negative-gradient identities gives

$$-\sum_i (F_i^W)^2 - \sum_{i,j} G_{ij}^2 \leq 0.$$

This closes the unprojected algebraic joint descent core of the companion Hebbian Kuramoto claim. It does not formalise projection onto a constrained coupling polytope, projected-gradient flow, KKT stationarity, or trajectory convergence.

2.6 Connections.lean

The connections module proves two simple but useful relationships. First, with zero weight regularisation, the Hebbian Lyapunov function is exactly the weighted Kuramoto potential under the full double-sum convention used in the library:

$$L_{K,0}(\theta, W) = V_W(\theta).$$

```

lemma hebbianL_zero_lam_eq_weighted
  (K : ℝ) (N : ℕ) (W : Fin N → Fin N → ℝ) (_hW : ∀ i j, W i j = W j i)
  (θ : Fin N → ℝ) :
  hebbianL K θ N W θ = weightedKuramotoV K N W θ

```

No factor of 1/2 appears in this statement, because both definitions use the same full double sum. Earlier informal descriptions using an upper-triangular convention would require a different normalisation.

Second, when $\lambda > 0$, the weight landscape is strictly convex in each individual weight coordinate at fixed phase:

$$\frac{\partial^2}{\partial W_{ij}^2} L_{K,\lambda}(\theta, W) > 0.$$

```

lemma hebbianL_convex_weight_entry
  (K lam : ℝ) (hlam : 0 < lam) (N : ℕ) (W : Fin N → Fin N → ℝ)
  (θ : Fin N → ℝ) (i j : Fin N) :
  0 < deriv (fun x => deriv (fun y => hebbianL K lam N (hebbianUpdateWeight W i j y) θ) x)
    (W i j)

```

The proof reuses the weight-gradient identity to identify the first derivative as an affine function $\lambda x - (K/2) \cos(\theta_j - \theta_i)$, then differentiates once more.

2.7 WitnessGeometry.lean

The witness-geometry module contains a small direct algebraic theorem about asymmetric quadratic restoring forces. It does not prove a theorem about the derivative of a piecewise-defined potential. Instead, it proves that if two positive curvatures a and b are unequal, then equal displacements on the two sides of a minimum produce unequal force magnitudes:

$$|-2a(-x)| \neq |-2bx| \quad \text{for } a, b, x > 0, a \neq b.$$

```

theorem barrier_asymmetry_direct
  (a b : ℝ) (ha : 0 < a) (hb : 0 < b) (hab : a ≠ b) (x : ℝ) (hx : 0 < x) :
  |(-2 * a * (-x))| ≠ |(-2 * b * x)|

```

The proof reduces the absolute values using the signs of the two expressions and then uses arithmetic to show that equality would imply $a = b$, contradicting the hypothesis. The result is included as a minimal formal bridge to the witness-theory interpretation of asymmetric potentials; it is not used in the Kuramoto proofs.

3. Verified Artifacts

The repository is available at:

<https://github.com/velvetmonkey/kuramoto-lean>

The artifact reviewed for this paper had commit:

484ccb075c61e6335667a32f976fae355f40cd6c

The Lean toolchain is pinned to:

leanprover/lean4:v4.31.0-rc1

Mathlib is pinned in lakefile.toml to:

971b90233bf92f8f8ac41f236bcac871e13b9f8e

The library root is:

```

-- Kuramoto synchronisation library: root import
import Kuramoto.OrderParameter
import Kuramoto.GradientFlow

```

```
import Kuramoto.Contraction
import Kuramoto.Weighted
import Kuramoto.Hebbian
import Kuramoto.Connections
import Kuramoto.WitnessGeometry
```

The verification commands are:

```
lake exe cache get
lake build
rg "sorry|admit|axiom" Kuramoto/
```

At the reviewed commit, `lake build` succeeds, and the `rg` command returns no matches. The theorem-bearing Lean modules contain 696 lines in `Kuramoto/*.lean`, with 14 public theorem/lemma results.

4. Development Methodology

The library was developed with an explicit `#check-before-cite` discipline. Before a Mathlib lemma name was used in a proof, the name was checked against the pinned Mathlib version. Names that failed to elaborate were not used. This rule is simple, but it addresses a common failure mode in AI-assisted Lean development: plausible API names that do not exist in the local version of Mathlib.

The importance of this rule became visible during development. Several expected names were absent or unsuitable under the pinned toolchain, and some informal theorem statements required correction after Lean exposed a sign or normalisation issue. For example, the general pairwise contraction expression must use a subtraction to represent the contribution to relative velocity; using a plus sign would cancel to zero under symmetry. Similarly, the zero-regularisation connection between `hebbianL` and `weightedKuramotoV` is equality, not equality up to a factor of two, because both definitions use full double sums.

A second methodological constraint was scoped target selection. Each proof attempt was framed around a concrete theorem statement and a proof strategy before proof search began. This reduced the search space and kept the development focused on finite algebraic identities rather than broader ODE claims. The result is a library of narrow but reliable facts: derivative identities, sign inequalities, and finite-sum descent statements.

Version pinning is also essential. Lean and Mathlib APIs evolve quickly, especially around analysis, derivatives, finite sums, and coercions. The pinned `lean-toolchain` and Mathlib revision make the artifact reproducible. A theorem that compiles against this commit has a precise dependency context; a theorem that merely appears plausible in prose does not.

Finally, zero-sorry verification should be interpreted correctly. The absence of `sorry`, `admit`, and new axioms means that the stated Lean theorems are fully

checked from the imported foundations. It does not mean that the theorems imply more than they state. In particular, this library proves algebraic gradient and Lyapunov-descent identities. It does not prove existence of ODE trajectories, convergence along trajectories, global synchronisation, or constrained projected dynamics.

5. Connections

5.1 Companion `flywheel-universe` project

The companion project `flywheel-universe` studies budgeted Hebbian Kuramoto dynamics for Max-Cut under coupling-resource constraints. It includes an experimental benchmark suite and a live browser demo:

- Project: <https://github.com/velvetmonkey/flywheel-universe>
- Live demo: <https://velvetmonkey.github.io/flywheel-universe/>
- Companion paper record: <https://zenodo.org/records/20303914>

The companion paper frames the Hebbian rule as a robustness primitive for heterogeneous-amplitude oscillator arrays rather than as a state-of-the-art Max-Cut solver. Its model includes symmetric weights, fixed support, row-sum budgets, and projection onto a constrained coupling set. The Lean library here does not formalise that full constrained projected system. Instead, `hebbian_joint_lyapunov_descent` proves the unprojected algebraic descent core:

$$dL[F, G] \leq 0$$

for the finite-dimensional phase-plus-weight vector field defined in the library. This is the part of the argument where the phase gradient identity and the weight gradient identity combine into a shared Lyapunov descent statement. Projection, tangent cones, Moreau decompositions, and KKT stationarity remain outside the present formalisation.

5.2 Witness-theory framing

The witness-theory connection is interpretive rather than a dependency of the Lean proofs. In that programme, observer-like coherence is modelled as descent on a loss or potential landscape. The Kuramoto gradient identity formalises one concrete instance of this pattern: local phase interactions can be represented as negative-gradient dynamics of a global potential. The Hebbian extension adds a second adaptive layer: weights as well as phases descend a joint Lyapunov function. The Lean contribution is to make this descent structure precise in a finite algebraic setting.

5.3 ATLAS and Mathlib context

The ATLAS project ([facebookresearch/atlas-lean](https://github.com/facebookresearch/atlas-lean)) is a large autoformalised Lean corpus. In exploratory work for this library, its Fourier-analysis material, especially the AddCircle scaffolding, was relevant as background for phase-circle reasoning. However, ATLAS did not supply the finite-dimensional Kuramoto dynamics or Lyapunov machinery needed here. The useful lesson is not that ATLAS is inadequate, but that domain-specific dynamical-systems libraries still require targeted formalisation beyond general mathematical scaffolding.

Mathlib itself provides the foundations used throughout this library: finite sums, complex numbers, real trigonometry, derivatives, norms, and order reasoning. The proofs here repeatedly exercise patterns that could be useful as future library contributions, especially around coordinate updates with `Function.update`, derivatives of finite sums, and finite-dimensional vector-field notation.

5.4 Related Kuramoto formalisation work

The most directly related Lean repository is [taejun-song/kuramoto-lean](https://github.com/taejun-song/kuramoto-lean), described on GitHub as a machine-checked proof of Kuramoto stability. Local inspection shows files devoted to complex Ott–Antonsen dynamics and continuum rigidity. That project and this one are complementary. The former addresses continuum/Ott–Antonsen stability structure; this library addresses finite- N algebraic identities, weighted finite networks, and Hebbian phase-plus-weight descent.

The broader Kuramoto literature includes the original synchronisation model, order-parameter analysis, Ott–Antonsen reductions, stability theory, and adaptive-coupling variants. This paper does not attempt to survey that literature exhaustively. Its focus is the Lean artifact and the proof-engineering methodology needed to formalise a finite subset of that theory.

6. Limitations and Future Work

The main limitation is that the library proves algebraic identities, not trajectory-level dynamical theorems. A directional Lyapunov inequality such as

$$\sum_i F_i(\theta) \partial_i V(\theta) \leq 0$$

is the algebraic core of a stability argument, but it is not itself a theorem about solutions of an ODE. To turn it into a trajectory statement, one must formalise existence and uniqueness of solutions, differentiability of the trajectory, the chain rule along the trajectory, and the appropriate invariance or compactness hypotheses. Those steps are intentionally outside the present library.

Several future directions are natural.

First, ODE existence and uniqueness for the finite Kuramoto vector field should be formalised directly from Mathlib’s Picard–Lindelof infrastructure. This would connect the algebraic vector-field definitions to actual solution curves.

Second, Lyapunov stability along trajectories should be proved once the ODE layer exists. The existing descent identities are designed to be reusable in that setting.

Third, full synchronisation convergence remains open. The current contraction results are local algebraic statements: exact $N = 2$ contraction and direct pair-coupling contribution for general N . They do not imply global convergence of all phases.

Fourth, phase-diameter monotonicity would require differentiating a maximum or diameter functional. This points toward a Dini-derivative or nonsmooth-analysis lemma that was not available in the local Mathlib context during development.

Fifth, the companion Hebbian Max-Cut model includes projection onto a constrained coupling set. Formalising projected constrained dynamics would require tangent-cone, normal-cone, variational-inequality, and KKT machinery beyond the current file set.

Sixth, the Hebbian phase-plus-weight descent connects naturally to associative-memory dynamics. Reading the adaptive weights W as a learned coupling makes a synchronised phase configuration an attractor selected by the weight field — the same mechanism that underlies Hopfield associative memory and its statistical-mechanics storage-capacity analysis (Amit, Gutfreund & Sompolinsky, 1985). The modern continuous-state Hopfield network, and its established equivalence to transformer attention (Ramsauer et al., 2021), suggests a route from the present finite- N descent identities toward formal retrieval- and capacity-level guarantees for attention-like dynamics. Formalising the storage-capacity and retrieval-stability side of this picture is a natural, if substantial, extension and is not attempted here.

Finally, the development suggests possible Mathlib contributions: reusable lemmas for coordinate derivatives through `Function.update`, finite-dimensional gradient notation over `Fin N`, and templates for differentiating nested finite sums of trigonometric expressions.

7. Conclusion

`kuramoto-lean` is a sorry-free Lean 4 foundation for finite- N Kuramoto and Hebbian phase-plus-weight dynamics. It formalises order-parameter boundedness, uniform and weighted gradient identities, contraction sign facts, Lyapunov descent identities, Hebbian weight and phase gradient identities, and a small witness-geometry force-asymmetry result. The companion `flywheel-universe` project supplies the experimental and modelling context; this library supplies the proof spine. More broadly, the project demonstrates that AI-assisted Lean development can produce clean, reproducible, sorry-free artifacts rapidly when

paired with strict API verification, scoped theorem targets, and honest boundaries around what has and has not been proved.

References and Links

- `kuramoto-lean` repository: <https://github.com/velvetmonkey/kuramoto-lean>
- Paper DOI: <https://doi.org/10.5281/zenodo.20468619>
- Paper record: <https://zenodo.org/records/20468619>
- Companion `flywheel-universe` repository: <https://github.com/velvetmonkey/flywheel-universe>
- Companion Max-Cut paper record: <https://zenodo.org/records/20303914>
- Live Hebbian Kuramoto demo: <https://velvetmonkey.github.io/flywheel-universe/>
- Lean theorem prover: <https://lean-lang.org/>
- Mathlib: <https://github.com/leanprover-community/mathlib4>
- ATLAS Lean repository: <https://github.com/facebookresearch/atlas-lean>
- `taejun-song/kuramoto-lean`: <https://github.com/taejun-song/kuramoto-lean>

Cited works

- Amit, D. J., Gutfreund, H., & Sompolinsky, H. (1985). Spin-glass models of neural networks. *Physical Review A*, 32(2), 1007–1018. <https://doi.org/10.1103/PhysRevA.32.1007>
- Ramsauer, H., Schäfl, B., Lehner, J., Seidl, P., Widrich, M., Gruber, L., Holzleitner, M., Adler, T., Kreil, D., Kopp, M., Klambauer, G., Brandstetter, J., & Hochreiter, S. (2021). Hopfield Networks is All You Need. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2008.02217>